

Problema da Árvore Geradora com Número Mínimo de Rótulos

Trabalho Prático – DCC059 | Teoria dos Grafos

Rian César Quintanilha Isaac Dizolele Kapela João Vítor Assunção
dos Santos Welder Salvador Victor

Universidade Federal de Juiz de Fora (UFJF)

Julho de 2026

- 1 Divisão das Tarefas
- 2 Definição do Problema
- 3 Estrutura de Dados
- 4 Union-Find
- 5 Heurística para o MLSTP
- 6 Geração das Instâncias
- 7 Conclusão

Divisão das Tarefas

Atividade	Responsável
Estrutura do grafo (lista de adjacência, CRUD)	Welder e Vítor
Heurística gulosa para o MLSTP	Welder e Vítor
Union-Find, remoção de ciclos e implementação do MLSTP	Rian
Geração das 20 instâncias de teste	Isaac

Definição do Problema

Seja $G = (V, E)$ um grafo conexo não direcionado onde cada aresta $e \in E$ possui um rótulo $\ell(e) \in L$.

O **MLSTP** consiste em encontrar uma árvore geradora T que minimize o número de rótulos distintos:

$$\min |\{\ell(e) \mid e \in E_T\}|$$

Complexidade

NP-difícil — heurísticas são necessárias para instâncias grandes.

Aplicações

- Redes de comunicação com múltiplas tecnologias
- Redes de transporte
- Distribuição de energia
- Infraestrutura com restrições de compatibilidade

Estrutura de Dados: Grafo

Classe No (vértice)

- `id` — identificador
- `visitado` — flag para buscas
- `grau_entrada`, `grau_saida`
- `vector<No*> vizinhos`

Struct Edge

- `int u, v` — extremos
- `double label` — rótulo

Classe Graph

- `vector<No*> nos`
- `bool direcionado`

Representação

Lista de adjacência — os rótulos são armazenados na struct `Edge` e associados às arestas durante a construção do grafo.

Métodos do Grafo

Operações

- `insertVertex / removeVertex`
- `insertEdge / removeEdge`
- `existEdge / changeWeight`

Consultas

- `verifyAdjacency`
- `calculateDegree`
- `listNeighbors`

Algoritmos

- `constructGraphFromEdges()`
— constrói o grafo a partir de arestas rotuladas
- `cutCycles()` — remove ciclos via Union-Find
- `minimumLabelingSpanningTree`
— heurística principal do MLSTP

Union-Find (Disjoint Set Union)

`find(x)`

Retorna a raiz do conjunto de x .

Otimização: path compression — aponta o nó direto para a raiz, achatando a árvore em chamadas futuras.

`unite(x, y)`

Une os conjuntos de x e y .

Retorna false se já pertencem ao mesmo conjunto $\Rightarrow$ **ciclo detectado**.

Otimização: union by rank — une a árvore menor sob a maior.

Uso no projeto

`cutCycles()`

Percorre as arestas do subgrafo gerado pela heurística e remove as que formariam ciclo, garantindo que o resultado final seja uma árvore geradora válida.

Complexidade

Quase linear: $O(m \cdot \alpha(n))$, onde α é a função inversa de Ackermann.

Heurística Gulosa para o MLSTP

Ideia central

A cada passo, escolher o rótulo que cobre o **maior número de vértices ainda não cobertos**.

Passos

- 1 Agrupar arestas por rótulo.
- 2 Enquanto houver vértices não cobertos:
 - Calcular o ganho de cada rótulo.
 - Selecionar o de maior ganho.
 - Inserir suas arestas no subgrafo.
- 3 Chamar `cutCycles()` para eliminar ciclos.

Resultado

Subgrafo $\tilde{G} = (V, \tilde{E}, \tilde{L})$ onde:

- V — todos os vértices
- $\tilde{L}$ — rótulos selecionados
- $\tilde{E}$ — arestas que formam a árvore geradora

Característica

Heurística gulosa — não garante ótimo global, mas produz soluções de boa qualidade com baixo custo computacional.

Procedimento de Geração (Xiong, 2005)

Parâmetros de entrada

- n — número de vértices
- ℓ — número de rótulos disponíveis
- *seed* — garante reprodutibilidade

Passos

- 1 Calcular m (número de arestas da instância)
- 2 Gerar **árvore geradora aleatória** com n vértices
→ conecta todos com $n - 1$ arestas
- 3 Sortear arestas adicionais sem repetição até completar m
- 4 Atribuir rótulo aleatório em $[1, \ell]$ a cada aresta

Por que a árvore geradora primeiro?

Sorteio puramente aleatório de m pares pode deixar vértices isolados.

A árvore garante conexidade com $n - 1$ arestas. As demais arestas adicionam ciclos, mas nunca desconectam o grafo.

Reprodutibilidade

Usando `std::mt19937` com *seed* fixa, a mesma *seed* sempre gera a mesma instância.

As 20 Instâncias Geradas

Instância	n	ℓ	m
01 – 02	100	25	990
03 – 04	100	25	2.475
05	100	25	3.960
06 – 07	200	50	3.980
08 – 09	200	50	9.950
10	200	50	15.920
11 – 12	500	125	24.950
13 – 14	500	125	62.375
15	500	125	99.800
16 – 17	1000	250	99.900
18 – 19	1000	250	249.750
20	1000	250	399.600

Critério de variação

- 4 tamanhos de grafo:
 $n \in \{100, 200, 500, 1000\}$
- 4 tamanhos de rótulos:
 $\ell \in \{25, 50, 125, 250\}$
- 2 seeds por combinação

Escala

De 990 arestas (instância pequena, esparsa) até 399.600 arestas (instância grande, densa).

O que foi entregue

- Estrutura de grafo completa em lista de adjacência
- Union-Find com path compression e union by rank
- Heurística gulosa para o MLSTP
- Gerador de instâncias seguindo Xiong (2005)
- 20 instâncias de teste cobrindo diferentes tamanhos e densidades

Obrigado!

Dúvidas?

Repositório

`https://github.com/isaac-kapela/
trabalho-final-grafos`

Relatório

`https://www.overleaf.com/project/
6a4adfd6087b1cdc622e98cc`